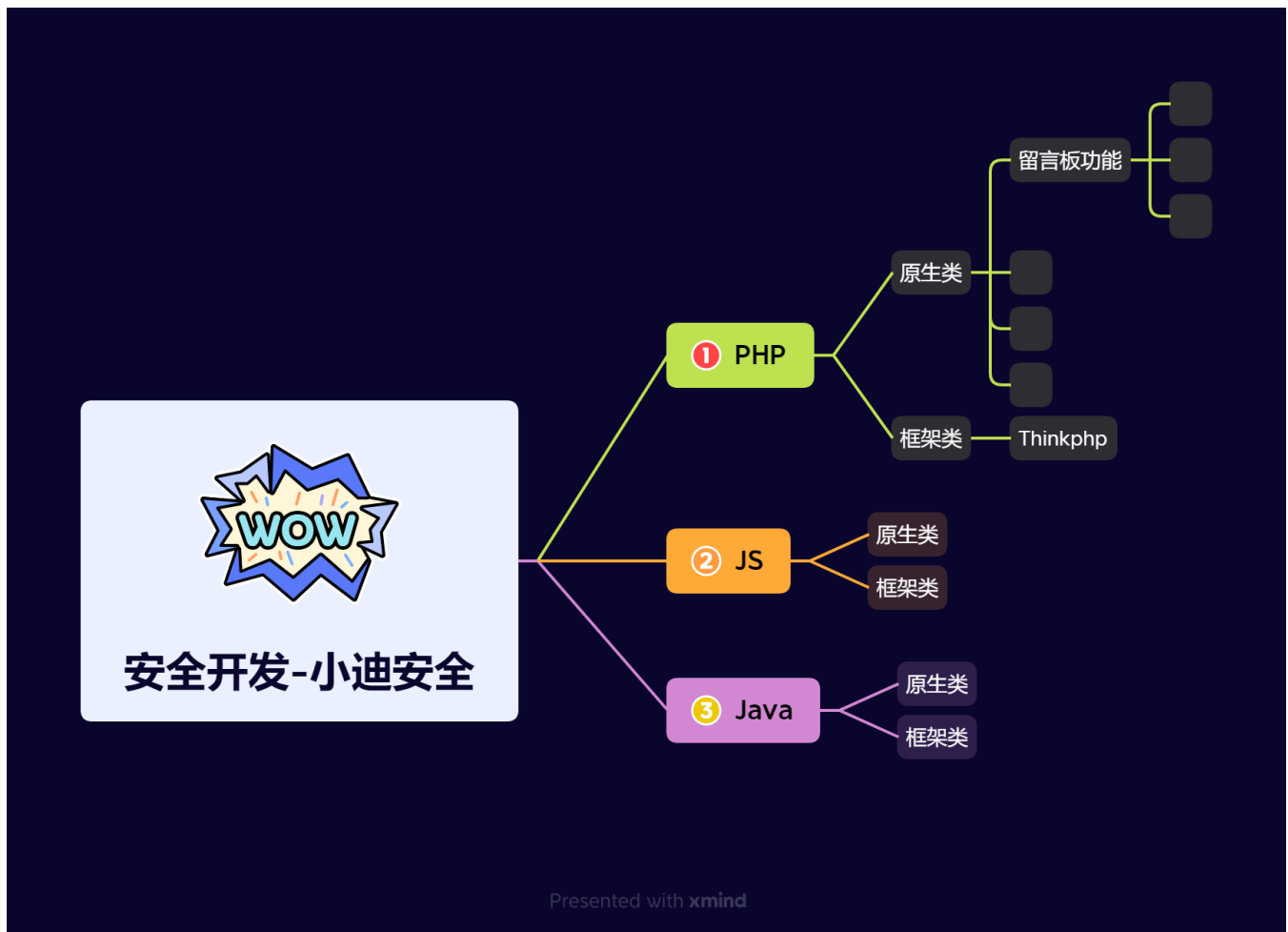


Day29 安全开发-JS应用&DOM树&加密编码库&断点调试&逆向分析&元素属性操作



1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

-
- 1、PHP后台身份验证模块实现
 - 2、Cookie&Session技术&差异
 - 3、Token数据包唯一性应用场景
 - 项目1：用cookie做后台身份验证
 - 项目2：用session做后台身份验证
 - 项目3：用token做用户登录判断
-

- 1、PHP文件管理-显示&上传功能实现
 - 2、文件上传-\$_FILES&过滤机制实现
 - 3、文件显示-目录遍历&过滤机制实现
-

- 1、PHP文件管理-下载&删除功能实现
 - 2、PHP文件管理-编辑&包含功能实现
-

- 1、PHP新闻显示-数据库操作读取显示
 - 2、PHP模版引用-自写模版&Smarty渲染
 - 3、PHP模版安全-RCE代码执行&三方漏洞
-

- 1、TP框架-开发-路由访问&数据库&文件上传&MVC模型
 - 2、TP框架-安全-不合规写法&内置过滤绕过&版本安全漏洞
-

- 1、JS应用-原生态开发&第三库开发
 - 2、JS功能-文件上传&登录验证&商品购买
-

- 1、JS技术-DOM树操作及安全隐患
 - 2、JS技术-加密编码及数据安全调试
-

2.演示案例

2.1 JS原生开发-DOM树-用户交互

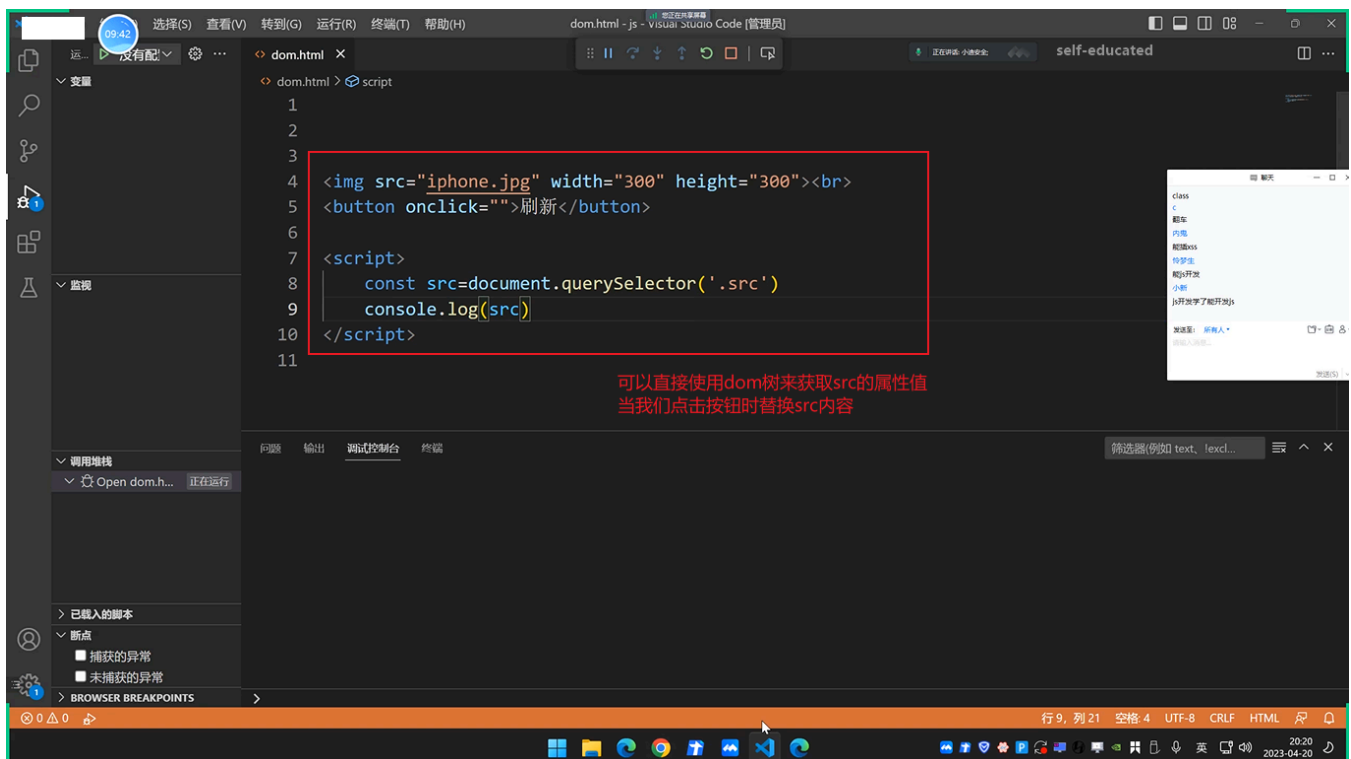
- 
- 1 DOM: 文档操作对象
 - 2 浏览器提供的一套专门用来操作网页代码内容的功能，实现自主或用户交互动作反馈
 - 3
 - 4 安全问题：本身的前端代码通过DOM技术实现代码的更新修改，但是更新修改如果修改的数据可以由用户来指定，就会造成DOM-XSS攻击！
 - 5 1、获取对象
 - 6 标签：直接写
 - 7 class: 加上符号.
 - 8 id: 加上符号#
 - 9 <h1 id="myHeader" onclick="getValue()">这是标题</h1>
 - 10 document.querySelector('h1')
 - 11 document.querySelector('#id')
 - 12 document.querySelector('#myHeader')
 - 13
 - 14 2、获取对象属性

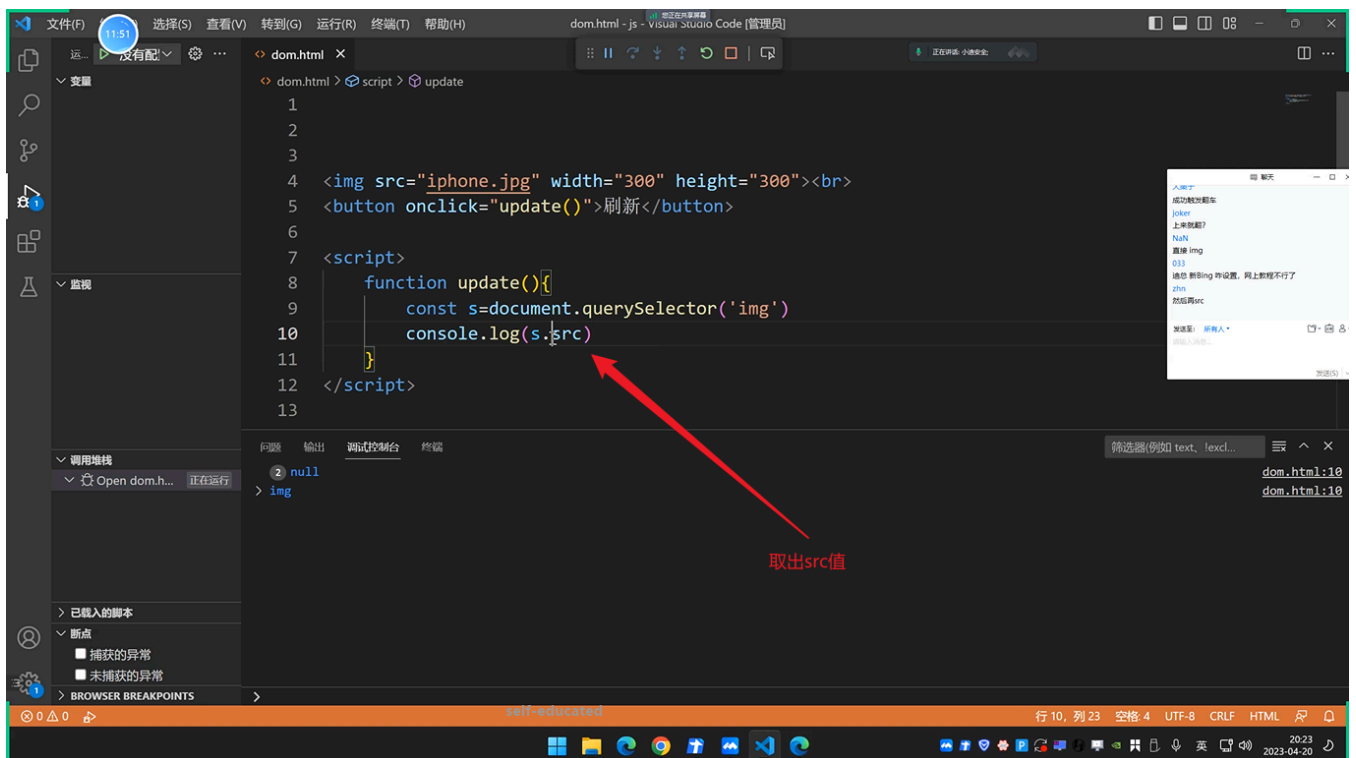
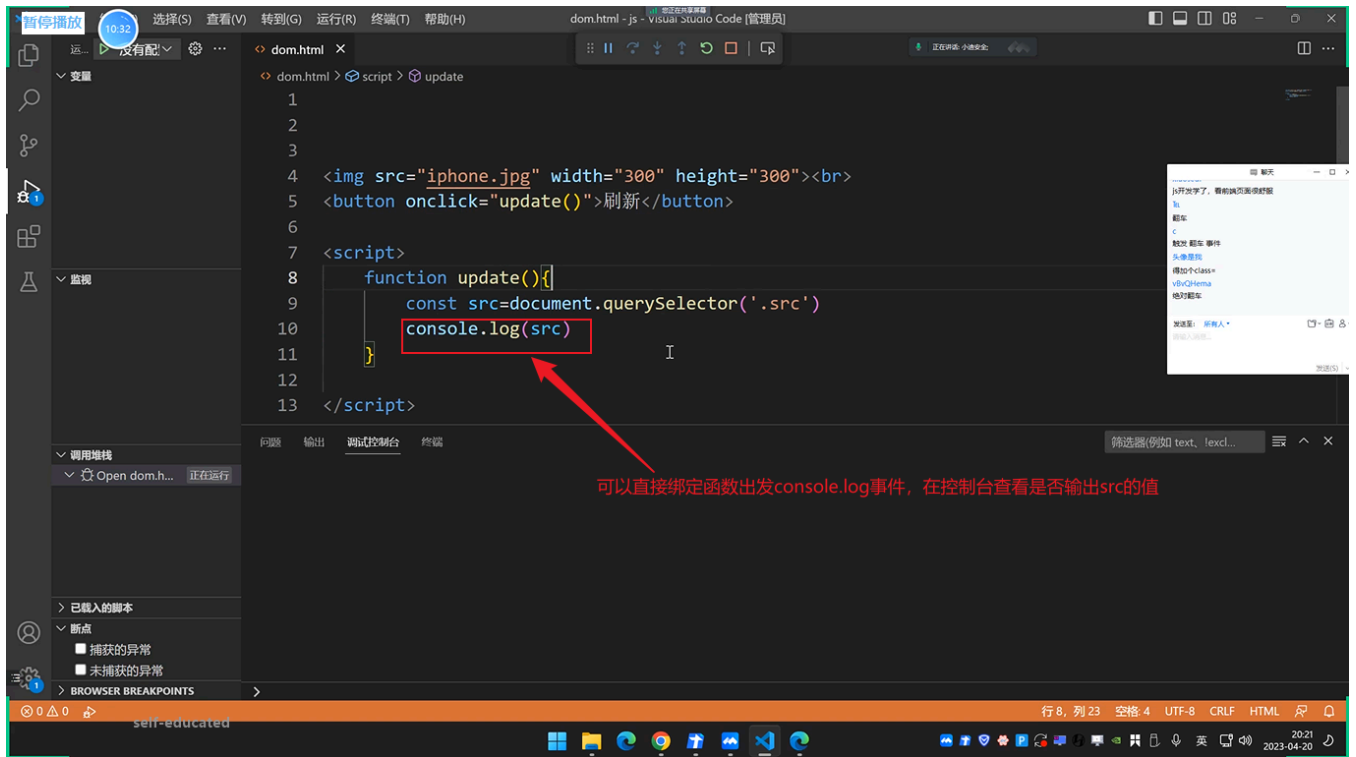
```

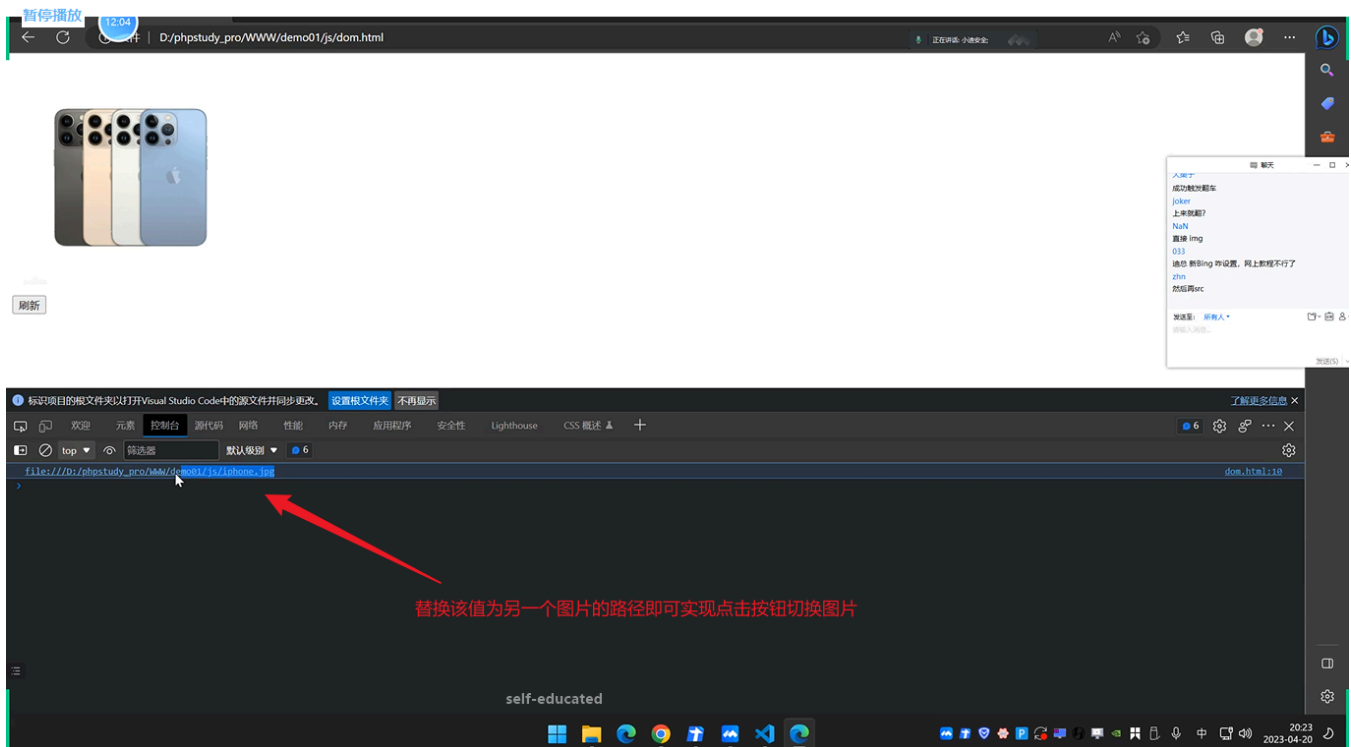
15 <h1 id="myHeader" onclick="getValue()">这是标题</h1>
16 const h1=document.querySelector('h1')
17 const id=h1.id
18 console.log(id)
19
20 2、操作元素数据
21 innerHTML 解析后续代码
22 innerText 不解析后续代码
23
24 3、操作元素属性
25 className src id等
26 </img>
27 const src=document.querySelector('img')
28 src.src='huawei.png'

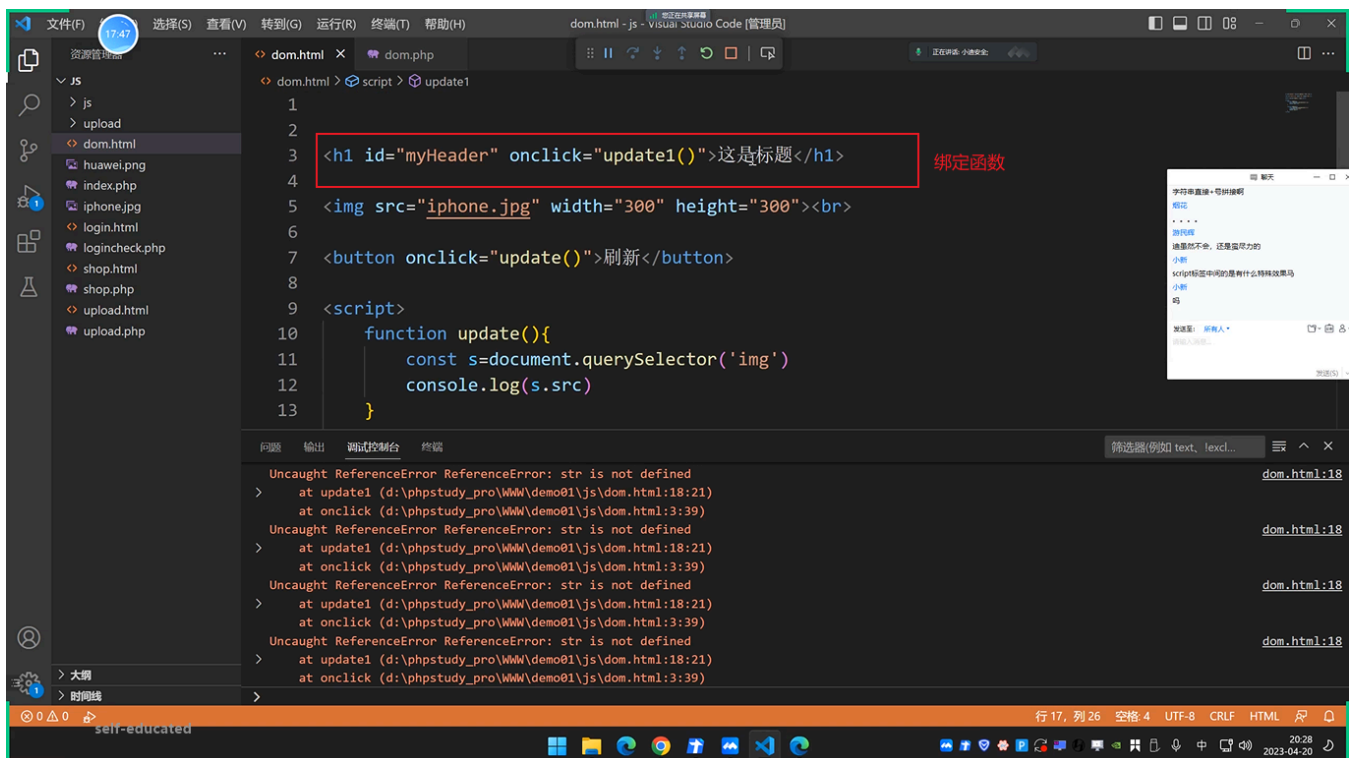
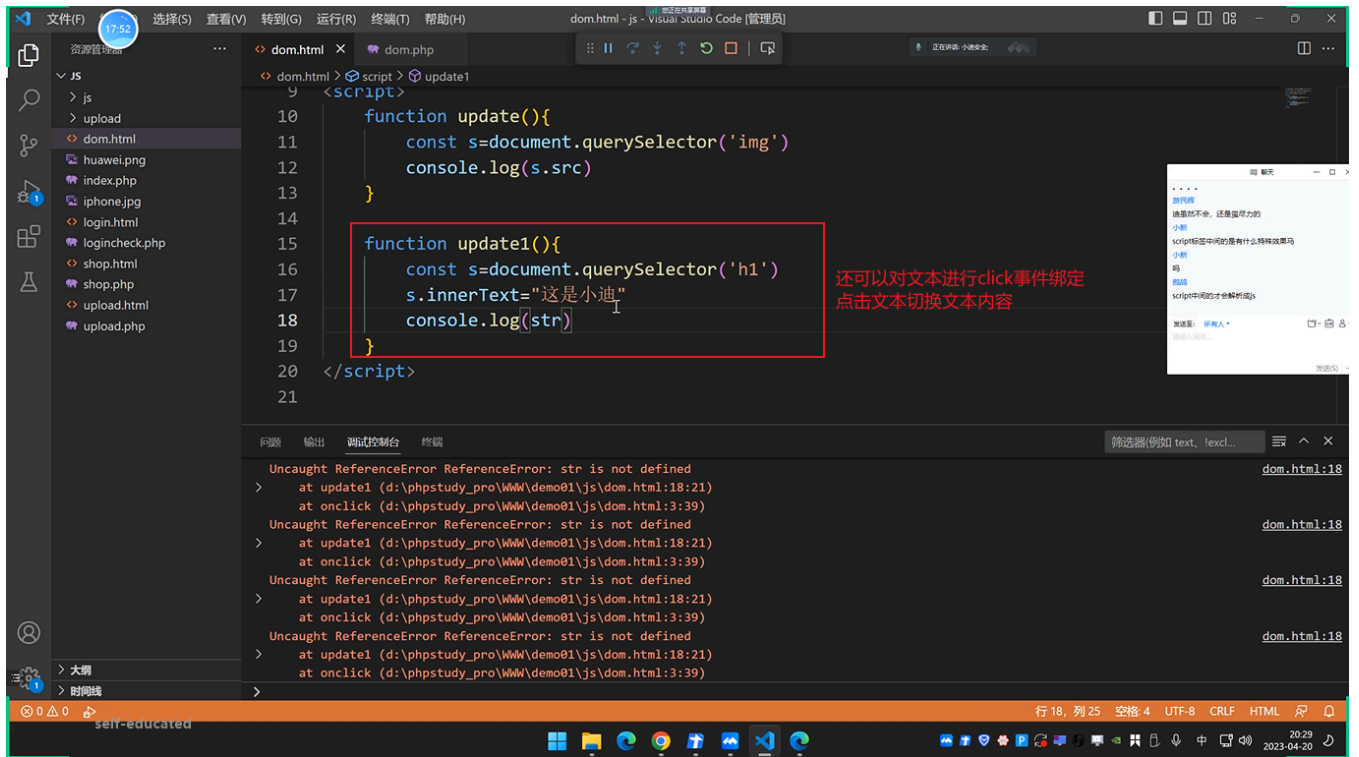
```

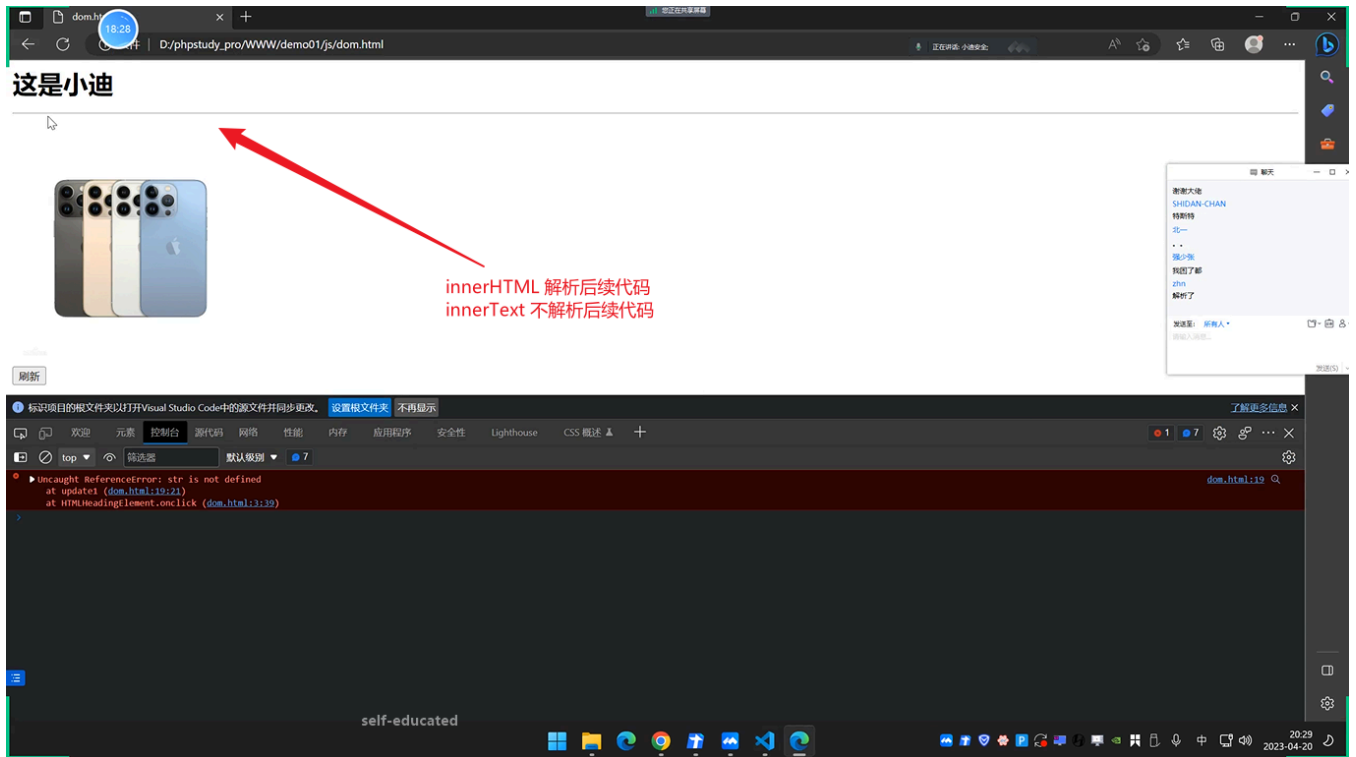
(1) DOM树:



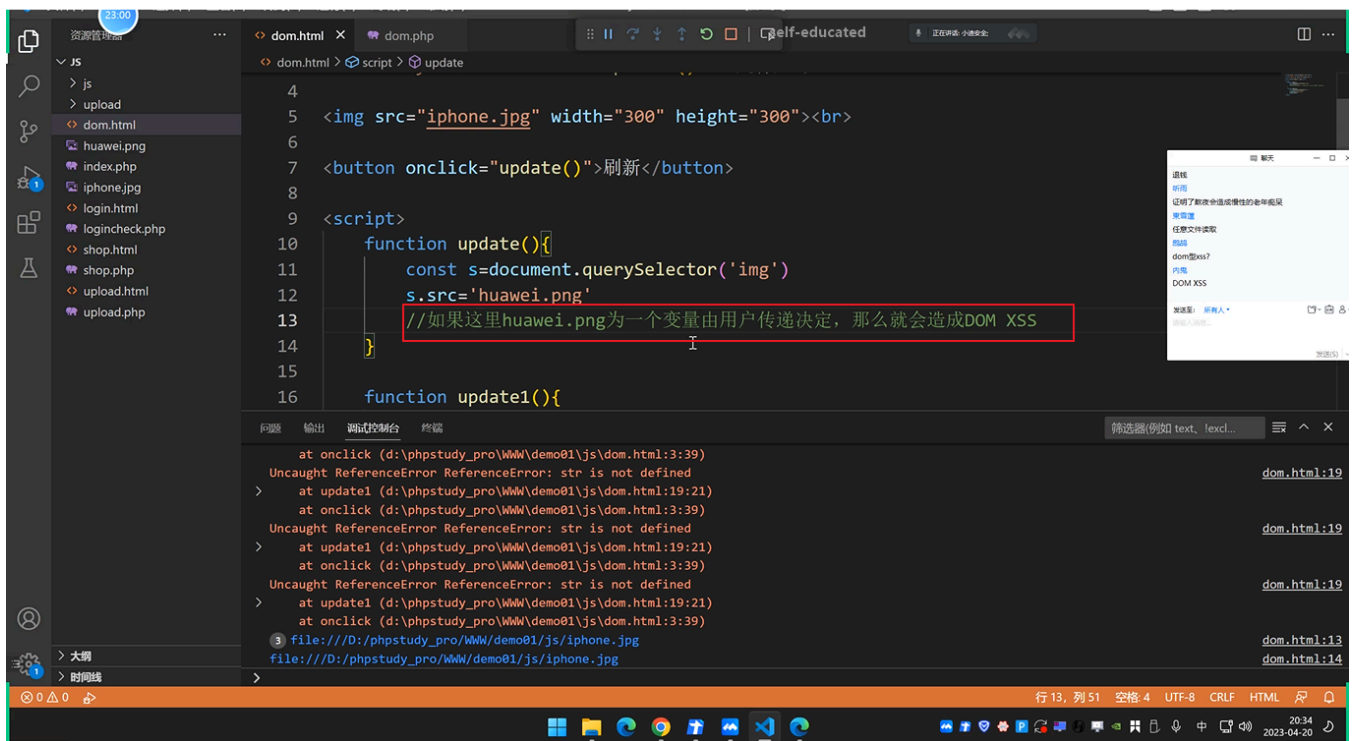


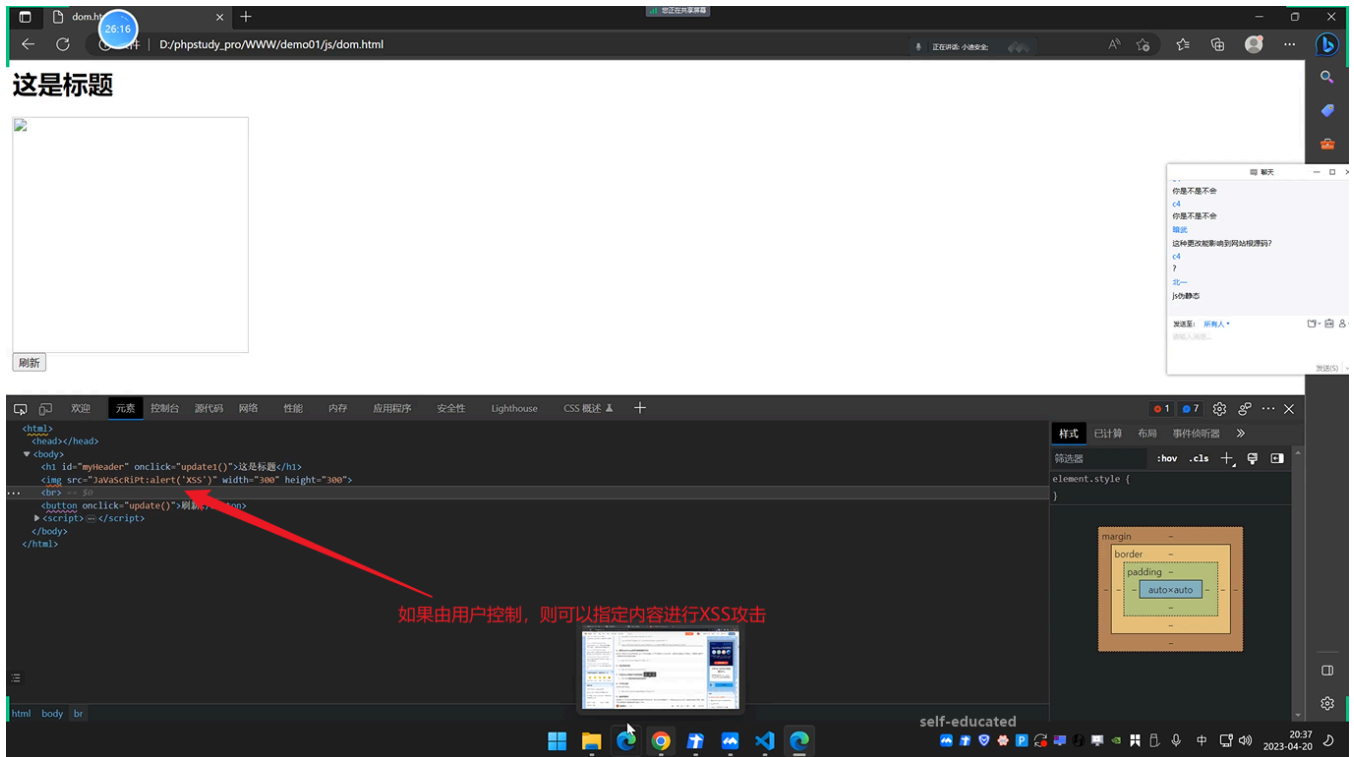






(2) 存在的问题:





2.2 JS导入库开发-编码加密-逆向调试

```
1  //Base64
2
3  //MD5
4  <script src="js/md5.js"></script>
5  <script>
6      var str1 = 'xiaodisec'
7      var str_encode = md5(str1);
8      console.log(str_encode)
9  </script>
10
11 //SHA1
12 <!DOCTYPE html>
13 <script src="js/crypto-js.js"></script>
14 <script>
15     var str1 = 'xiaodisec';
16     var str_encode = CryptoJS.SHA1(str1).toString(); // 注意: 1是数字1
17     console.log(str_encode)
18 </script>
19 </html>
20
21 //HMAC
22 <script src="js/crypto-js.js"></script>
23 <script>
24     var key = 'key';
25     var str1 = 'xiaodisec';
26     var hash = CryptoJS.HmacSHA256(key, str1);
```



```

27     var str_encode = CryptoJS.enc.Hex.stringify(hash);
28     console.log(str_encode)  //
    '11a7960cd583ee2c3f1ed910dbc3b6c3991207cbc527d122f69e84d13cc5ce5c'
29 </script>
30
31
32 //AES
33 <script src="js/crypto-js.js"></script>
34     <script type="text/javascript">
35         var aseKey = "12345678"      // 定制密钥，长度必须为：8/16/32位，长度不一致也没问题
36         var message = "xiaodisec";  // 需要加密的内容
37         // 加密 DES/AES切换只需要修改 CryptoJS.AES <=> CryptoJS.DES
38         var encrypt = CryptoJS.AES.encrypt(message,
CryptoJS.enc.Utf8.parse(aseKey),  // 参数1=密钥，参数2=加密内容
39             {
40                 mode: CryptoJS.mode.ECB, // 为DES的工作方式
41                 padding: CryptoJS.pad.Pkcs7 // 当加密后密文长度达不到指定整数倍(8个字节、
16个字节)则填充对应字符
42             }
43         ).toString(); // toString=转字符串类型
44
45         console.log(encrypt); // 在弹窗中打印字符串 2vcsEDJv9vAZZLgFLjkZ9A==
46
47         //解密
48         var decrypt = CryptoJS.AES.decrypt(encrypt,
CryptoJS.enc.Utf8.parse(aseKey), // 参数1=密钥，参数2=解密内容
49             {
50                 mode: CryptoJS.mode.ECB,
51                 padding: CryptoJS.pad.Pkcs7
52             }
53         ).toString(CryptoJS.enc.Utf8); // toString=转字符串类型,并指定编码
54         console.log(decrypt); // "xiaodisec"
55     </script>
56
57 //DES
58 <script src="js/crypto-js.js"></script>
59     <script type="text/javascript">
60         var aseKey = "12345678"      // 定制密钥，长度必须为：8/16/32位，长度不一致也没问题
61         var message = "xiaodisec";  // 需要加密的内容
62         // 加密 DES/AES切换只需要修改 CryptoJS.AES <=> CryptoJS.DES
63         var encrypt = CryptoJS.DES.encrypt(message, CryptoJS.enc.Utf8.parse(aseKey),
// 参数1=密钥，参数2=加密内容
64             {
65                 mode: CryptoJS.mode.ECB, // 为DES的工作方式
66                 padding: CryptoJS.pad.Pkcs7 // 当加密后密文长度达不到指定整数倍(8个字节、16个字
节)则填充对应字符
67             }
68         ).toString(); // toString=转字符串类型

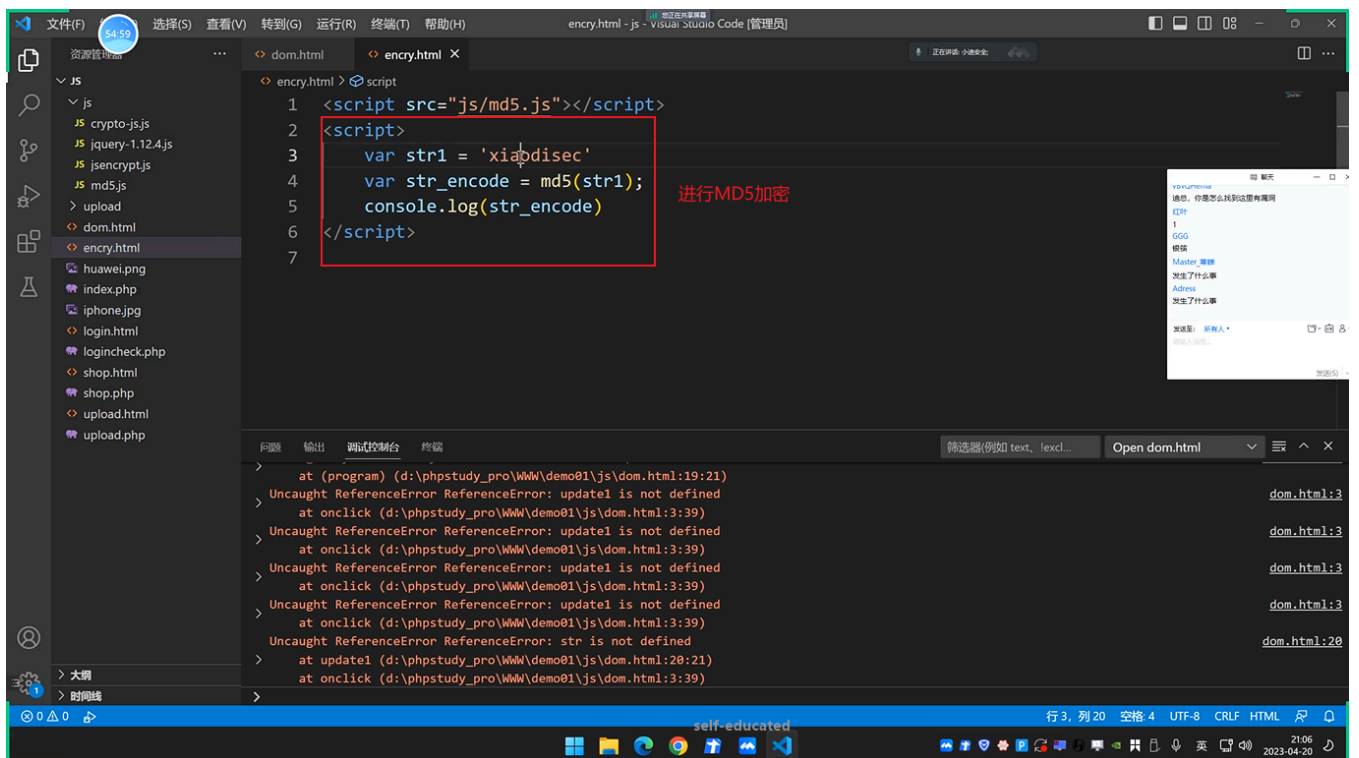
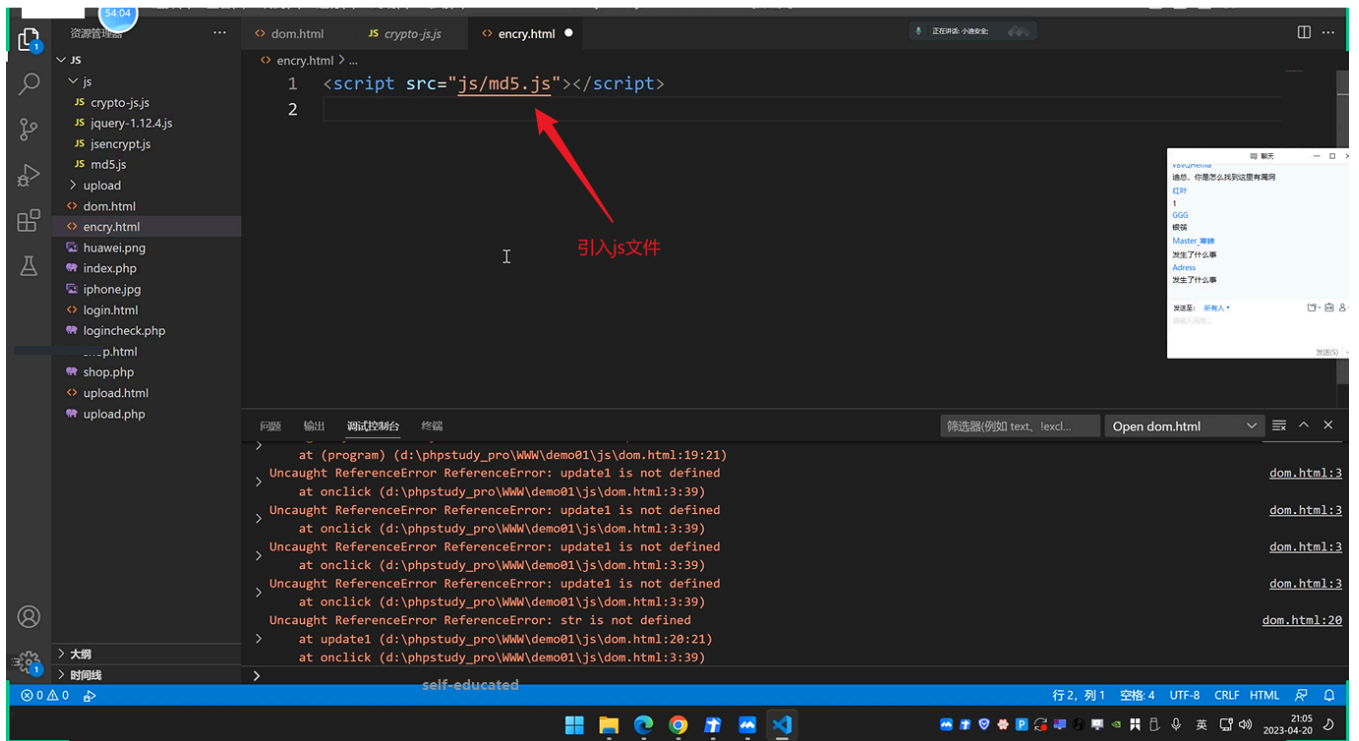
```

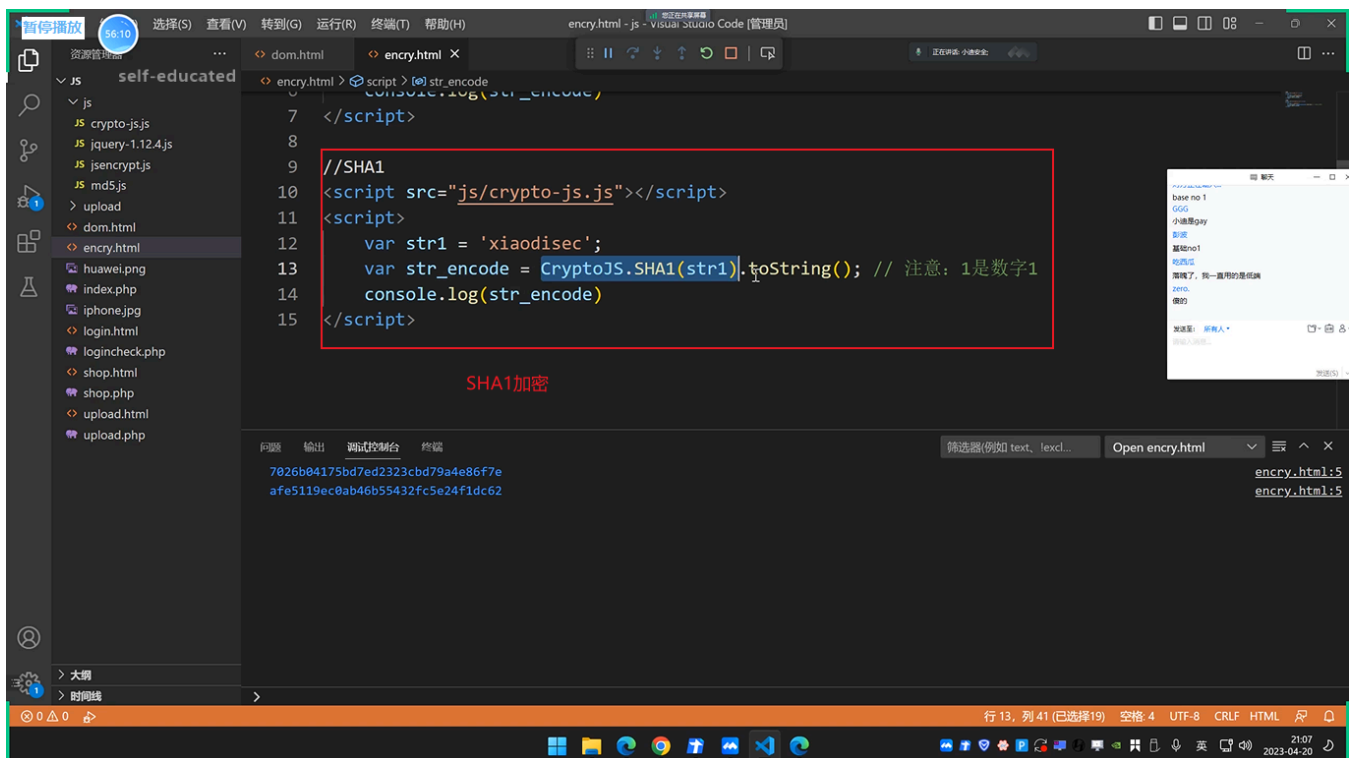
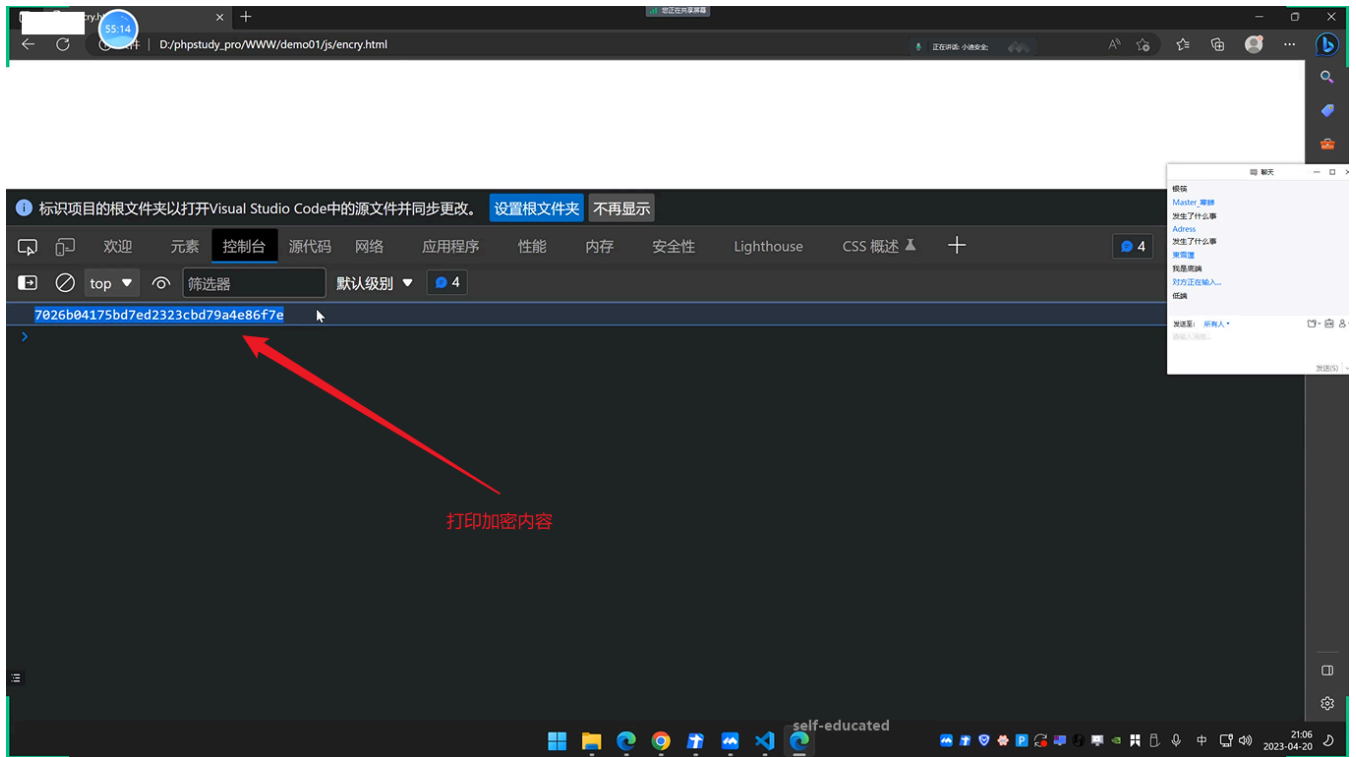
```

69
70     console.log(encrypt); // 控制台打印 CDVNwmEwDRM
71
72     //解密
73     var decrypt = CryptoJS.DES.decrypt(encrypt, CryptoJS.enc.Utf8.parse(aseKey),
// 参数1=密钥, 参数2=解密内容
74     {
75         mode: CryptoJS.mode.ECB,
76         padding: CryptoJS.pad.Pkcs7
77     }
78     ).toString(CryptoJS.enc.Utf8); // toString=转字符串类型,并指定编码
79     console.log(decrypt); // 控制台打印 "i am xiaozhou ?"
80 </script>
81
82
83 //RSA
84 <script src="js/jsencrypt.js"></script>
85     <script type="text/javascript">
86         // 公钥 私匙是通过公匙计算生成的, 不能盲目设置
87         var PUBLIC_KEY = '-----BEGIN PUBLIC KEY-----
MFwwDQYJKoZIhvcNAQEBBQADSwAwSAJBALyBJ6kZ/VFJYTV3vOC07jqwIqgyvHu1v6us/8wz1SBqQ2+eOTX7
s5zKfXY40yZWDoCaIGk+tP/sc0D6dQzjaxECAwEAAQ==-----END PUBLIC KEY-----';
88         //私钥
89         var PRIVATE_KEY = '-----BEGIN PRIVATE KEY-----
MIIBVQIBADANBgkqhkiG9w0BAQEFAASCAT8wggE7AgEAAKEAvIEnqRn9Uu1hNXe84LTuOpYiqPK8e6w/q6z/
zDOVIGpDb545NfuznMp9djjTJlYogJogaT60/+xzQpP1DONrEQIDAQABAKEAu7DFsqQEDDnKJpiwYfUE9ysI
IWNTNLJWZDN/Bu2dYIV4D02A5aHZfMe48rga5Bkowq2LAL1Y3tqsOFte3M6yoQIhAOSfSAU3H6jION1EiZab
UrVGqiFLCb5Ut3Jz9NN+5p59AiEA0xQDMrxWBBJ9BYq6RRY4pXwa/MthX/8Hy+3GnvNw/yUCIG/3Ee578KVY
akq5pih8KSveVj037C2qj60d30k3XPqBAiEAqGPvxTsAuBDz0kCBIPqASGzArum1jkrLsoHHkakOfu0CIDuh
xKQWH1XFD079ppYAPcVO3bph672qGD84YUaHF+pQ-----END PRIVATE KEY-----';
90         //使用公钥加密
91         var encrypt = new JSEncrypt(); //实例化加密对象
92         encrypt.setPublicKey(PUBLIC_KEY); //设置公钥
93         var message = 'xiaodisec' // 需要加密的数据
94         var encrypted = encrypt.encrypt(message); //对指定数据进行加密
95         console.log(encrypted) //
'JQ83h8tmJpsSZcb4BJ3eQvuqIAS3ejepCUUnoFhQEvm8fA8bf1Y/fG+D01bSIVNJF6EOZKe4wa0njv6aOa
r9w=='
96         //使用私钥解密
97         var decrypt = new JSEncrypt(); // 创建解密对象
98         decrypt.setPrivateKey(PRIVATE_KEY); //设置私钥
99         var uncrpyted = decrypt.decrypt(encrypted); //解密 'xiaodisec'
100         console.log(uncrpyted);
101     </script>

```

(1) JS导入库开发:





```
15 </script>
16
17 //HMAC
18 <script src="js/crypto-js.js"></script>
19 <script>
20     var key = 'key';
21     var str1 = 'xiaodisec';
22     var hash = CryptoJS.HmacSHA256(key, str1);
23     var str_encode = CryptoJS.enc.Hex.stringify(hash);
24     console.log(str_encode) // '11a7960cd583ee2c3f1ed910dbc3b6c3991207cbc527d122'
25 </script>
26
```

Output in console:

```
7026b04175bd7ed2323cbd79a4e86f7e
afe5119ec0ab46b55432fc5e24f1dc62
afe5119ec0ab46b55432fc5e24f1dc62
ce22eaa1c5ebd3dfb3f4474b66f6d3612d4cb3ee
```

```
33 var message = 'xiaodisec'; // 需要加密的字符串
34 // 加密 DES/AES切换只需要修改 CryptoJS.AES <=> CryptoJS.DES
35 var encrypt = CryptoJS.AES.encrypt(message, CryptoJS.enc.Utf8.parse(aseKey), // 参
36 {
37     mode: CryptoJS.mode.CFB, // 为DES的工作方式
38     padding: CryptoJS.pad.Pkcs7 // 当加密后密文长度达不到指定整数倍(8个
39 }).toString(); // toString=转字符串类型
40
41 console.log(encrypt);
42 </script>
```

Output in console:

```
7026b04175 /article
afe5119ec0 /aside
afe5119ec0 /audio
ce22eaa1c5 /b
afe5119ec0 /base
08ac6dc877 /bdi
afe5119ec0 /bdo
ce22eaa1c5 /blockquote
08ac6dc877
```

- 1 得知加密方法后，进行逆向调试，进行逆向分析。